Core Data Structures in Python

Lists, Implementation, Searching, and Hash Mappings

Martin Benning

University College London
ENGF0034 – Design and Professional Skills

20 October 2026



Lecture Overview

Objectives

To master Python's core collection types (`list`, `dict`, `set`), understand the reference-based memory model and aliasing, inspect how the standard Python interpreter (**CPython**) implements dynamic arrays, and compare linear, binary, and hash-based lookup strategies.

Part I: Lists, Memory & Internals

- Python Lists: Indexing, Slicing & Mutation

Lecture Overview

Objectives

To master Python's core collection types (`list`, `dict`, `set`), understand the reference-based memory model and aliasing, inspect how the standard Python interpreter (**CPython**) implements dynamic arrays, and compare linear, binary, and hash-based lookup strategies.

Part I: Lists, Memory & Internals

- Python Lists: Indexing, Slicing & Mutation
- Memory Model: Names, References & Aliasing

Lecture Overview

Objectives

To master Python's core collection types (`list`, `dict`, `set`), understand the reference-based memory model and aliasing, inspect how the standard Python interpreter (**CPython**) implements dynamic arrays, and compare linear, binary, and hash-based lookup strategies.

Part I: Lists, Memory & Internals

- Python Lists: Indexing, Slicing & Mutation
- Memory Model: Names, References & Aliasing
- Shallow Copy (`.copy()`) vs. Deep Copy

Lecture Overview

Objectives

To master Python's core collection types (`list`, `dict`, `set`), understand the reference-based memory model and aliasing, inspect how the standard Python interpreter (**CPython**) implements dynamic arrays, and compare linear, binary, and hash-based lookup strategies.

Part I: Lists, Memory & Internals

- Python Lists: Indexing, Slicing & Mutation
- Memory Model: Names, References & Aliasing
- Shallow Copy (`.copy()`) vs. Deep Copy
- Under the Hood: CPython & Dynamic Arrays

Lecture Overview

Objectives

To master Python's core collection types (`list`, `dict`, `set`), understand the reference-based memory model and aliasing, inspect how the standard Python interpreter (**CPython**) implements dynamic arrays, and compare linear, binary, and hash-based lookup strategies.

Part I: Lists, Memory & Internals

- Python Lists: Indexing, Slicing & Mutation
- Memory Model: Names, References & Aliasing
- Shallow Copy (`.copy()`) vs. Deep Copy
- Under the Hood: CPython & Dynamic Arrays

Part II: Complexity, Search & Hashing

- Big-$\mathcal{O}$ Notation & Performance ($\mathcal{O}(1)$ vs. $\mathcal{O}(N)$)

Lecture Overview

Objectives

To master Python's core collection types (`list`, `dict`, `set`), understand the reference-based memory model and aliasing, inspect how the standard Python interpreter (**CPython**) implements dynamic arrays, and compare linear, binary, and hash-based lookup strategies.

Part I: Lists, Memory & Internals

- Python Lists: Indexing, Slicing & Mutation
- Memory Model: Names, References & Aliasing
- Shallow Copy (`.copy()`) vs. Deep Copy
- Under the Hood: CPython & Dynamic Arrays

Part II: Complexity, Search & Hashing

- Big-$\mathcal{O}$ Notation & Performance ($\mathcal{O}(1)$ vs. $\mathcal{O}(N)$)
- Geometric Growth & Amortised Analysis

Lecture Overview

Objectives

To master Python's core collection types (`list`, `dict`, `set`), understand the reference-based memory model and aliasing, inspect how the standard Python interpreter (**CPython**) implements dynamic arrays, and compare linear, binary, and hash-based lookup strategies.

Part I: Lists, Memory & Internals

- Python Lists: Indexing, Slicing & Mutation
- Memory Model: Names, References & Aliasing
- Shallow Copy (`.copy()`) vs. Deep Copy
- Under the Hood: CPython & Dynamic Arrays

Part II: Complexity, Search & Hashing

- Big-$\mathcal{O}$ Notation & Performance ($\mathcal{O}(1)$ vs. $\mathcal{O}(N)$)
- Geometric Growth & Amortised Analysis
- Searching Lists: Linear ($\mathcal{O}(N)$) vs. Binary ($\mathcal{O}(\log N)$)

Lecture Overview

Objectives

To master Python's core collection types (`list`, `dict`, `set`), understand the reference-based memory model and aliasing, inspect how the standard Python interpreter (**CPython**) implements dynamic arrays, and compare linear, binary, and hash-based lookup strategies.

Part I: Lists, Memory & Internals

- Python Lists: Indexing, Slicing & Mutation
- Memory Model: Names, References & Aliasing
- Shallow Copy (`.copy()`) vs. Deep Copy
- Under the Hood: CPython & Dynamic Arrays

Part II: Complexity, Search & Hashing

- Big-$\mathcal{O}$ Notation & Performance ($\mathcal{O}(1)$ vs. $\mathcal{O}(N)$)
- Geometric Growth & Amortised Analysis
- Searching Lists: Linear ($\mathcal{O}(N)$) vs. Binary ($\mathcal{O}(\log N)$)
- Hash Mappings: `dict` & `set` ($\mathcal{O}(1)$ Lookup)

Python Lists: Interface and Usage

Data Structure of the Week: The List

Motivation

Engineering applications constantly manage collections of data (sensor readings, floor requests, user records). The Python `list` is the primary built-in sequence for this.

Data Structure of the Week: The List

Motivation

Engineering applications constantly manage collections of data (sensor readings, floor requests, user records). The Python `list` is the primary built-in sequence for this.

Definition

A Python `list` is a **mutable**, **ordered**, **dynamic sequence**:

- **Ordered:** Elements maintain a defined sequence, accessible by integer position (**index**).

Week 1 `str` vs. Week 2 `list`

Property	`str`	`list`
Ordered sequence?	Yes	Yes
Indexing / Slicing?	Yes	Yes
Element types	Characters	Any object
Mutable?	**No**	**Yes**

Data Structure of the Week: The List

Motivation

Engineering applications constantly manage collections of data (sensor readings, floor requests, user records). The Python `list` is the primary built-in sequence for this.

Definition

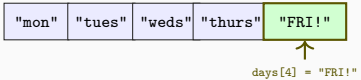
A Python `list` is a **mutable**, **ordered**, **dynamic sequence**:

- **Ordered:** Elements maintain a defined sequence, accessible by integer position (**index**).
- **Mutable:** Elements can be replaced, inserted, or removed *in place* after creation.

Week 1 `str` vs. Week 2 `list`

Property	`str`	`list`
Ordered sequence?	Yes	Yes
Indexing / Slicing?	Yes	Yes
Element types	Characters	Any object
Mutable?	**No**	**Yes**

In-Place Mutation Allowed:



Data Structure of the Week: The List

Motivation

Engineering applications constantly manage collections of data (sensor readings, floor requests, user records). The Python `list` is the primary built-in sequence for this.

Definition

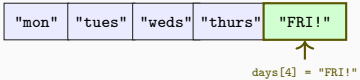
A Python `list` is a **mutable**, **ordered**, **dynamic sequence**:

- **Ordered:** Elements maintain a defined sequence, accessible by integer position (**index**).
- **Mutable:** Elements can be replaced, inserted, or removed *in place* after creation.
- **Dynamic & Heterogeneous:** Can grow or shrink at runtime and hold any mix of Python objects.

Week 1 `str` vs. Week 2 `list`

Property	`str`	`list`
Ordered sequence?	Yes	Yes
Indexing / Slicing?	Yes	Yes
Element types	Characters	Any object
Mutable?	**No**	**Yes**

In-Place Mutation Allowed:



Creating, Indexing, and Slicing Lists

Syntax, Indexing & Slicing (`interactive_lists.py`)

Lists use square brackets `[]`. Indexing is **zero-based** (0 to len − 1) and slices `L[start:stop]` include `start` but exclude `stop`.

```
days = ["mon", "tues", "weds", "thurs", "fri"]
count = len(days)       # 5 (number of elements)

# Zero-based indexing (0 to len-1)
first = days[0]         # "mon"
day2  = days[1]         # "tues" (second day!)

# Negative indexing & Slicing [start:stop]
last_day = days[-1]     # "fri"
midweek  = days[1:4]    # ["tues", "weds", "thurs"]
```

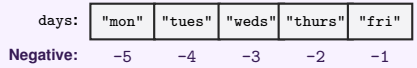
Creating, Indexing, and Slicing Lists

Syntax, Indexing & Slicing (`interactive_lists.py`)

Lists use square brackets `[]`. Indexing is **zero-based** (0 to len − 1) and slices `L[start:stop]` include `start` but exclude `stop`.

```
1  days = ["mon", "tues", "weds", "thurs", "fri"]
2  count = len(days)       # 5 (number of elements)
3
4  # Zero-based indexing (0 to len-1)
5  first = days[0]         # "mon"
6  day2  = days[1]         # "tues" (second day!)
7
8  # Negative indexing & Slicing [start:stop]
9  last_day = days[-1]     # "fri"
10 midweek  = days[1:4]    # ["tues", "weds", "thurs"]
```

Visualising List Indices



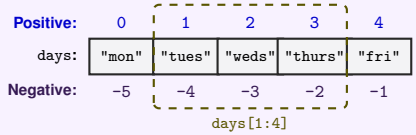
Creating, Indexing, and Slicing Lists

Syntax, Indexing & Slicing (`interactive_lists.py`)

Lists use square brackets `[]`. Indexing is **zero-based** (0 to len − 1) and slices `L[start:stop]` include `start` but exclude `stop`.

```
1  days = ["mon", "tues", "weds", "thurs", "fri"]
2  count = len(days)       # 5 (number of elements)
3
4  # Zero-based indexing (0 to len-1)
5  first = days[0]         # "mon"
6  day2  = days[1]         # "tues" (second day!)
7
8  # Negative indexing & Slicing [start:stop]
9  last_day = days[-1]     # "fri"
10 midweek  = days[1:4]    # ["tues", "weds", "thurs"]
```

Visualising List Indices



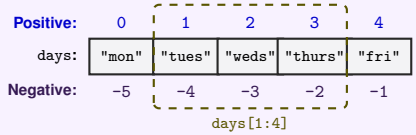
Creating, Indexing, and Slicing Lists

Syntax, Indexing & Slicing (`interactive_lists.py`)

Lists use square brackets `[]`. Indexing is **zero-based** (0 to len − 1) and slices `L[start:stop]` include `start` but exclude `stop`.

```
1  days = ["mon", "tues", "weds", "thurs", "fri"]
2  count = len(days)       # 5 (number of elements)
3
4  # Zero-based indexing (0 to len-1)
5  first = days[0]         # "mon"
6  day2  = days[1]         # "tues" (second day!)
7
8  # Negative indexing & Slicing [start:stop]
9  last_day = days[-1]     # "fri"
10 midweek  = days[1:4]    # ["tues", "weds", "thurs"]
```

Visualising List Indices



Common Off-by-One Trap

`days[2]` returns `"weds"` (the *third* element), and `days[5]` raises `IndexError: list index out of range`.

Modifying Lists: Mutability and Adding Elements

Item Assignment (interactive_lists2.py)

```
1  days = ["mon", "tues", "weds", "thurs", "fri"]
2  days[4] = "FRIDAY!!!" # Mutates index 4 in-place
```

Modifying Lists: Mutability and Adding Elements

Item Assignment (`interactive_lists2.py`)

```
1 days = ["mon", "tues", "weds", "thurs", "fri"]
2 days[4] = "FRIDAY!!!" # Mutates index 4 in-place
```

Adding Elements

- `.append(x)`: Adds item `x` to the **end**.

```
1 days.append("sat")
2 days.insert(0, "sun") # Insert at beginning
```

Modifying Lists: Mutability and Adding Elements

Item Assignment (`interactive_lists2.py`)

```
1 days = ["mon", "tues", "weds", "thurs", "fri"]
2 days[4] = "FRIDAY!!!" # Mutates index 4 in-place
```

Step-by-Step State Trace

Operation	Resulting `days`
Initial	`["mon", ..., "fri"]`
`days[4] = "FRI!"`	`["mon", ..., "FRI!"]`
`days.append("sat")`	`["mon", ..., "FRI!", "sat"]`
`days.insert(0,"sun")`	`["sun", "mon", ..., "sat"]`

Adding Elements

- `.append(x)`: Adds item `x` to the **end**.
- `.insert(i, x)`: Inserts `x` at index `i` (shifts items at indices $\geqslant$ `i` one slot right).

```
1 days.append("sat")
2 days.insert(0, "sun") # Insert at beginning
```

Modifying Lists: Mutability and Adding Elements

Item Assignment (`interactive_lists2.py`)

```
1 days = ["mon", "tues", "weds", "thurs", "fri"]
2 days[4] = "FRIDAY!!!" # Mutates index 4 in-place
```

Step-by-Step State Trace

Operation	Resulting `days`
Initial	`["mon", ..., "fri"]`
`days[4] = "FRI!"`	`["mon", ..., "FRI!"]`
`days.append("sat")`	`["mon", ..., "FRI!", "sat"]`
`days.insert(0,"sun")`	`["sun", "mon", ..., "sat"]`

Adding Elements

- `.append(x)`: Adds item `x` to the **end**.
- `.insert(i, x)`: Inserts `x` at index `i` (shifts items at indices $\geqslant$ `i` one slot right).
- `.extend(seq)`: Appends all items from `seq`.

```
1 days.append("sat")
2 days.insert(0, "sun") # Insert at beginning
```

`.append()` vs. `.extend()`

```
1 a = [1, 2]; a.append([3, 4]) # [1, 2, [3,
      4]]
2 b = [1, 2]; b.extend([3, 4]) # [1, 2, 3, 4]
```

Removing Elements from a List

Methods for Removal (`interactive_lists2.py`)

- `del L[i]`: Deletes item at index `i` (no return).

```
1  days = ["mon", "tues", "weds", "thurs", "fri"]
2  del days[0]           # ["tues", "weds", "thurs",
       "fri"]
3  last = days.pop()     # last == "fri"
```

Removing Elements from a List

Methods for Removal (`interactive_lists2.py`)

- `del L[i]`: Deletes item at index `i` (no return).
- `L.pop()`: Removes & **returns** the last item.
- `L.pop(i)`: Removes & **returns** item at index `i`.

```
1  days = ["mon", "tues", "weds", "thurs", "fri"]
2  del days[0]           # ["tues", "weds", "thurs",
       "fri"]
3  last = days.pop()     # last == "fri"
```

Removing Elements from a List

Methods for Removal (`interactive_lists2.py`)

- `del L[i]`: Deletes item at index `i` (no return).
- `L.pop()`: Removes & **returns** the last item.
- `L.pop(i)`: Removes & **returns** item at index `i`.
- `L.remove(v)`: Removes **first** match of value `v`.

```
1 days = ["mon", "tues", "weds", "thurs", "fri"]
2 del days[0]           # ["tues", "weds", "thurs",
      "fri"]
3 last = days.pop()     # last == "fri"
```

Comparison of Removal Operations

Syntax	Target	Returns?	Error
`del L[i]`	Index `i`	No	`IndexError`
`L.pop()`	Last idx	Yes	`IndexError`
`L.pop(i)`	Index `i`	Yes	`IndexError`
`L.remove(v)`	Value `v`	No	`ValueError`

Removing Elements from a List

Methods for Removal (`interactive_lists2.py`)

- `del L[i]`: Deletes item at index `i` (no return).
- `L.pop()`: Removes & **returns** the last item.
- `L.pop(i)`: Removes & **returns** item at index `i`.
- `L.remove(v)`: Removes **first** match of value `v`.

```
1 days = ["mon", "tues", "weds", "thurs", "fri"]
2 del days[0]          # ["tues", "weds", "thurs",
      "fri"]
3 last = days.pop()    # last == "fri"
```

Comparison of Removal Operations

Syntax	Target	Returns?	Error
`del L[i]`	Index `i`	No	`IndexError`
`L.pop()`	Last idx	Yes	`IndexError`
`L.pop(i)`	Index `i`	Yes	`IndexError`
`L.remove(v)`	Value `v`	No	`ValueError`

Important Note on Return Values

In-place mutating methods (`.append()`, `.insert()`, `.remove()`, `.sort()`) return `None`!
Only `.pop()` returns the removed item.

Iterating Over Lists Pythonically

1. Index Loop (`interactive_lists4.py`)

C-style iteration using `range(0, len(days))`:

```
1  days = ["mon", "tues", "weds"]
2  for day_num in range(0, len(days)):
3      print(days[day_num])
```

Iterating Over Lists Pythonically

1. Index Loop (`interactive_lists4.py`)

C-style iteration using `range(0, len(days))`:

```
1  days = ["mon", "tues", "weds"]
2  for day_num in range(0, len(days)):
3      print(days[day_num])
```

2. Direct Iteration (`src/lists/list.py`)

Directly iterates over elements without manual indexing:

```
1  for day in days:
2      print(day)
```

Iterating Over Lists Pythonically

1. Index Loop (interactive_lists4.py)

C-style iteration using `range(0, len(days))`:

```
days = ["mon", "tues", "weds"]
for day_num in range(0, len(days)):
    print(days[day_num])
```

2. Direct Iteration (src/lists/list.py)

Directly iterates over elements without manual indexing:

```
for day in days:
    print(day)
```

3. `enumerate()` (Index + Value)

When you need both position `i` and element `day`:

```
for i, day in enumerate(days):
    print(f"Index {i}: {day}")
# Output: Index 0: mon | Index 1: tues | ...
```

Iterating Over Lists Pythonically

1. Index Loop (`interactive_lists4.py`)

C-style iteration using `range(0, len(days))`:

```
days = ["mon", "tues", "weds"]
for day_num in range(0, len(days)):
    print(days[day_num])
```

2. Direct Iteration (`src/lists/list.py`)

Directly iterates over elements without manual indexing:

```
for day in days:
    print(day)
```

3. `enumerate()` (Index + Value)

When you need both position `i` and element `day`:

```
for i, day in enumerate(days):
    print(f"Index {i}: {day}")
# Output: Index 0: mon | Index 1: tues | ...
```

Rule of Thumb

- Elements only? Use `for x in L:`.
- Index + element? Use `enumerate(L)`.
- Mutating in-place? Use `range(len(L))`.

The Python Memory Model and References

Understanding Variables in Python

The “Variables as Boxes” Metaphor (C)

In languages like C, a variable is a fixed memory box at a specific address holding a raw value. Assigning `b = a` copies the value into a second box.

C Model (Boxes Holding Values):



Understanding Variables in Python

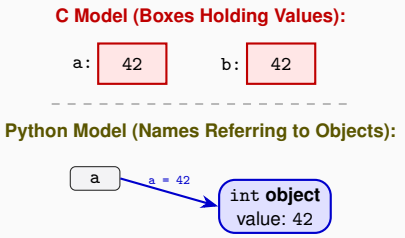
The “Variables as Boxes” Metaphor (C)

In languages like C, a variable is a fixed memory box at a specific address holding a raw value. Assigning `b = a` copies the value into a second box.

The “Variables as Labels” Metaphor (Python)

In Python, variables are **names** (labels) bound to **object references**:

- **The Heap:** Dynamically managed memory where Python stores objects of any size.



Understanding Variables in Python

The “Variables as Boxes” Metaphor (C)

In languages like C, a variable is a fixed memory box at a specific address holding a raw value. Assigning `b = a` copies the value into a second box.

The “Variables as Labels” Metaphor (Python)

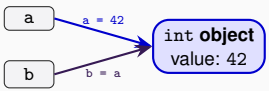
In Python, variables are **names** (labels) bound to **object references**:

- **The Heap:** Dynamically managed memory where Python stores objects of any size.
- **Reference (Pointer):** A memory address pointing to an object in the heap. Assignment (`b = a`) copies the *reference*, not the object!

C Model (Boxes Holding Values):



Python Model (Names Referring to Objects):



The Impact of References: Aliasing

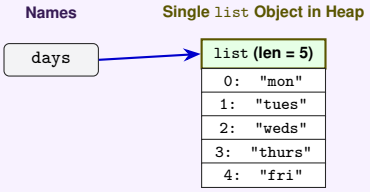
What is Aliasing? (`interactive_lists3.py`)

When two or more variable names refer to the **exact same mutable object** in memory, they are **aliases**.

- **Object Identity (`id(x)`):** Built-in function returning the unique integer memory address of object `x`.

```
1  # 1. Create a list object and bind 'days' to it
2  days = ["mon", "tues", "weds", "thurs", "fri"]
3
4  # 2. Bind 'moredays' to the SAME list object:
5  moredays = days
6  assert id(days) == id(moredays) # Same identity!
```

Namespace vs. Heap Object



The Impact of References: Aliasing

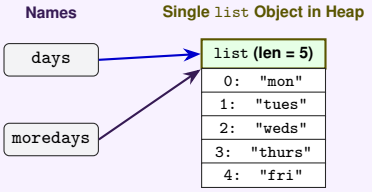
What is Aliasing? (`interactive_lists3.py`)

When two or more variable names refer to the **exact same mutable object** in memory, they are **aliases**.

- **Object Identity (`id(x)`):** Built-in function returning the unique integer memory address of object `x`.

```
1  # 1. Create a list object and bind 'days' to it
2  days = ["mon", "tues", "weds", "thurs", "fri"]
3
4  # 2. Bind 'moredays' to the SAME list object:
5  moredays = days
6  assert id(days) == id(moredays) # Same identity!
```

Namespace vs. Heap Object



The Consequences of Aliasing

1. Mutating Through an Alias

```
1  days = ["mon", "tues", "weds", "thurs", "FRIDAY
       !!!"]
2  moredays = days
3
4  # Modify the list via the 'moredays' alias
5  moredays[4] = "fri"
6
7  # What does print(days[4]) output?
8  print(days[4])
```

The Consequences of Aliasing

1. Mutating Through an Alias

```
1  days = ["mon", "tues", "weds", "thurs", "FRIDAY
       !!!"]
2  moredays = days
3
4  # Modify the list via the 'moredays' alias
5  moredays[4] = "fri"
6
7  # What does print(days[4]) output?
8  print(days[4])
```

Output: `"fri"` — `days` was mutated too!

The Consequences of Aliasing

1. Mutating Through an Alias

```
days = ["mon", "tues", "weds", "thurs", "FRIDAY
    !!!"]
moredays = days

# Modify the list via the 'moredays' alias
moredays[4] = "fri"

# What does print(days[4]) output?
print(days[4])
```

Output: `"fri"` — `days` was mutated too!

2. Passing Lists to Functions

Function parameters in Python are passed by **object reference** (aliasing the caller's argument):

```
def PopFirst(items):
    items.pop(0)  # Mutates caller's list!

workdays = ["mon", "tues", "weds"]
PopFirst(workdays)
print(workdays)   # ["tues", "weds"]
```

Work on a copy if the caller's list must not change!

How to Copy a List: Shallow Copying

Creating an Independent List (`interactive_lists3.py`)

To avoid aliasing, construct a new `list` object:

```
1  days = ["mon", "tues", "weds", "thurs", "fri"]
2  copy1 = days.copy()   # Method 1: .copy() (clear)
3  copy2 = days[:]       # Method 2: full slice
4
5  # Verification: mutate copy1 at index 0
6  copy1[0] = "YAWN!"
7  # copy1 is ["YAWN!", "tues", "weds", "thurs", "fri
       "]
8  # days  is ["mon",   "tues", "weds", "thurs", "fri
       "]
```

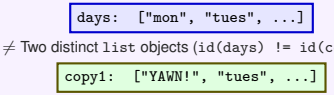
How to Copy a List: Shallow Copying

Creating an Independent List (`interactive_lists3.py`)

To avoid aliasing, construct a new `list` object:

```
1  days = ["mon", "tues", "weds", "thurs", "fri"]
2  copy1 = days.copy()    # Method 1: .copy() (clear)
3  copy2 = days[:]        # Method 2: full slice
4
5  # Verification: mutate copy1 at index 0
6  copy1[0] = "YAWN!"
7  # copy1 is ["YAWN!", "tues", "weds", "thurs", "fri
       "]
8  # days  is ["mon",   "tues", "weds", "thurs", "fri
       "]
```

Memory Layout After `days.copy()`



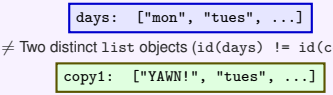
How to Copy a List: Shallow Copying

Creating an Independent List (`interactive_lists3.py`)

To avoid aliasing, construct a new `list` object:

```
1  days = ["mon", "tues", "weds", "thurs", "fri"]
2  copy1 = days.copy()    # Method 1: .copy() (clear)
3  copy2 = days[:]        # Method 2: full slice
4
5  # Verification: mutate copy1 at index 0
6  copy1[0] = "YAWN!"
7  # copy1 is ["YAWN!", "tues", "weds", "thurs", "fri
       "]
8  # days  is ["mon",   "tues", "weds", "thurs", "fri
       "]
```

Memory Layout After `days.copy()`



Why is it called a “Shallow” Copy?

`.copy()` and `[:]` only duplicate the **outermost** list container. If the list contains nested mutable objects, the inner lists are *still shared*!

Shallow Copy vs. Deep Copy: Nested Lists

The Shallow Copy Pitfall

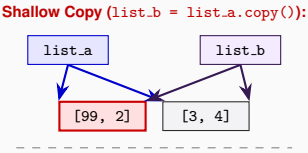
```
1  list_a = [[1, 2], [3, 4]]
2  list_b = list_a.copy()   # Shallow copy
3
4  # Mutate an inner list via list_b:
5  list_b[0][0] = 99
6  print(list_a)
```

Shallow Copy vs. Deep Copy: Nested Lists

The Shallow Copy Pitfall

```
1  list_a = [[1, 2], [3, 4]]
2  list_b = list_a.copy()   # Shallow copy
3
4  # Mutate an inner list via list_b:
5  list_b[0][0] = 99
6  print(list_a)
```

Output: `[[99, 2], [3, 4]]` — `list_a[0]` and `list_b[0]` point to the **same inner list**!

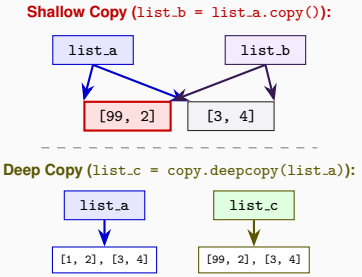


Shallow Copy vs. Deep Copy: Nested Lists

The Shallow Copy Pitfall

```
1  list_a = [[1, 2], [3, 4]]
2  list_b = list_a.copy()    # Shallow copy
3
4  # Mutate an inner list via list_b:
5  list_b[0][0] = 99
6  print(list_a)
```

Output: `[[99, 2], [3, 4]]` — `list_a[0]` and `list_b[0]` point to the **same inner list**!



The Fix: `copy.deepcopy()`

```
1  import copy
2  list_c = copy.deepcopy(list_a) # Clones all
       levels
```

List Implementation: Dynamic Arrays

Looking Under the Hood: CPython and Dynamic Arrays

What is CPython?

Python is the language specification; **CPython** is the standard, default Python interpreter (downloaded from `python.org`), written in **C**. Inspecting CPython's internal C structures reveals why some list operations are instantaneous while others slow down at scale.

Looking Under the Hood: CPython and Dynamic Arrays

What is CPython?

Python is the language specification; **CPython** is the standard, default Python interpreter (downloaded from `python.org`), written in **C**. Inspecting CPython's internal C structures reveals why some list operations are instantaneous while others slow down at scale.

The Underlying Structure: Dynamic Array

CPython implements a `list` as a **Dynamic Array**: a **contiguous block of memory** (adjacent, gap-free memory cells) that resizes automatically as elements are added.

Looking Under the Hood: CPython and Dynamic Arrays

What is CPython?

Python is the language specification; **CPython** is the standard, default Python interpreter (downloaded from `python.org`), written in **C**. Inspecting CPython's internal C structures reveals why some list operations are instantaneous while others slow down at scale.

The Underlying Structure: Dynamic Array

CPython implements a `list` as a **Dynamic Array**: a **contiguous block of memory** (adjacent, gap-free memory cells) that resizes automatically as elements are added.

The Heterogeneous Challenge

Fast array indexing requires every slot in the contiguous block to have the **exact same byte width**.

Yet a single Python list can hold objects of wildly different sizes:

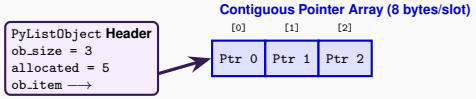
- `42` (`int`, 28 bytes)
- `"Hello World!"` (`str`, 61 bytes)
- `[1, 2, 3, 4, 5]` (nested `list`)

Question: How can differently-sized objects fit in uniform contiguous slots?

CPython Implementation: Contiguous Array of Pointers

The Solution: Array of Pointers

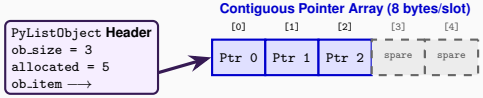
The list does not store the objects inside its contiguous buffer. Instead, it stores a contiguous array of **64-bit pointers** (uniform 8-byte memory addresses) referring to objects scattered on the heap.



CPython Implementation: Contiguous Array of Pointers

The Solution: Array of Pointers

The list does not store the objects inside its contiguous buffer. Instead, it stores a contiguous array of **64-bit pointers** (uniform 8-byte memory addresses) referring to objects scattered on the heap.



CPython `PyListObject` Metadata

1. `ob_size`: Active elements (`len(L)`).
2. `allocated`: Total capacity of pointer array.
3. `ob_item`: Pointer to contiguous array.

Key Insight: `allocated` $\geqslant$ `ob_size` (**Over-allocation**: spare slots reserved for growth).

CPython Implementation: Contiguous Array of Pointers

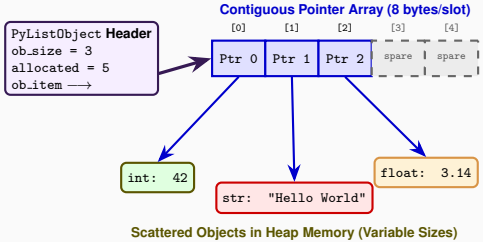
The Solution: Array of Pointers

The list does not store the objects inside its contiguous buffer. Instead, it stores a contiguous array of **64-bit pointers** (uniform 8-byte memory addresses) referring to objects scattered on the heap.

CPython `PyListObject` Metadata

1. `ob_size`: Active elements (`len(L)`).
2. `allocated`: Total capacity of pointer array.
3. `ob_item`: Pointer to contiguous array.

Key Insight: `allocated` $\geqslant$ `ob_size` (**Over-allocation**: spare slots reserved for growth).



Performance Analysis and Amortisation

Measuring Efficiency: Big-$\mathcal{O}$ Notation ($\mathcal{O}(f(N))$)

What is Big-$\mathcal{O}$ Notation?

Let $N = \text{len}(L)$ be the number of elements. **Big-$\mathcal{O}$ notation**, written $\mathcal{O}(f(N))$, describes how an operation's runtime (or step count) **scales asymptotically** as N grows large, ignoring machine-dependent constants:

- $\mathcal{O}(1)$ **(Constant):** Runtime is independent of N.

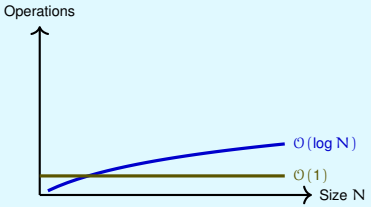
Measuring Efficiency: Big-$\mathcal{O}$ Notation ($\mathcal{O}(f(N))$)

What is Big-$\mathcal{O}$ Notation?

Let $N = \text{len}(L)$ be the number of elements. **Big-$\mathcal{O}$ notation**, written $\mathcal{O}(f(N))$, describes how an operation's runtime (or step count) **scales asymptotically** as N grows large, ignoring machine-dependent constants:

- $\mathcal{O}(1)$ **(Constant):** Runtime is independent of N.
- $\mathcal{O}(\log N)$ **(Logarithmic):** Grows with $\log_2 N$ (halving the problem each step; for $N = 10^6$, $\log_2 N \approx 20$).

Comparing Growth Rates as N Grows



Notation: $\Longrightarrow$ means "implies"; $\approx$ means "approximately equal to".

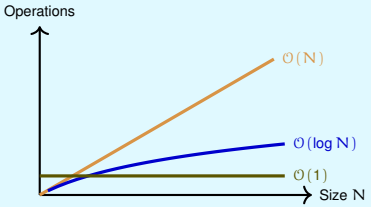
Measuring Efficiency: Big-$\mathcal{O}$ Notation ($\mathcal{O}(f(N))$)

What is Big-$\mathcal{O}$ Notation?

Let $N = \text{len}(L)$ be the number of elements. **Big-$\mathcal{O}$ notation**, written $\mathcal{O}(f(N))$, describes how an operation's runtime (or step count) **scales asymptotically** as N grows large, ignoring machine-dependent constants:

- $\mathcal{O}(1)$ **(Constant):** Runtime is independent of N.
- $\mathcal{O}(\log N)$ **(Logarithmic):** Grows with $\log_2 N$ (halving the problem each step; for $N = 10^6$, $\log_2 N \approx 20$).
- $\mathcal{O}(N)$ **(Linear):** Grows directly in proportion to N.

Comparing Growth Rates as N Grows



Notation: $\Longrightarrow$ means "implies"; $\approx$ means "approximately equal to".

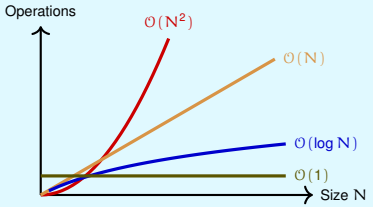
Measuring Efficiency: Big-$\mathcal{O}$ Notation ($\mathcal{O}(f(N))$)

What is Big-$\mathcal{O}$ Notation?

Let $N = \text{len}(L)$ be the number of elements. **Big-$\mathcal{O}$ notation**, written $\mathcal{O}(f(N))$, describes how an operation's runtime (or step count) **scales asymptotically** as N grows large, ignoring machine-dependent constants:

- $\mathcal{O}(1)$ **(Constant):** Runtime is independent of N.
- $\mathcal{O}(\log N)$ **(Logarithmic):** Grows with $\log_2 N$ (halving the problem each step; for $N = 10^6$, $\log_2 N \approx 20$).
- $\mathcal{O}(N)$ **(Linear):** Grows directly in proportion to N.
- $\mathcal{O}(N^2)$ **(Quadratic):** Grows with N^2 (e.g. repeating an $\mathcal{O}(N)$ operation N times).

Comparing Growth Rates as N Grows



Notation: $\Longrightarrow$ means "implies"; $\approx$ means "approximately equal to".

Analysing Performance: $\mathcal{O}(1)$ Constant-Time Operations

Why Indexing and Length are $\mathcal{O}(1)$

- **Indexing (`L[i]`):** Because pointers are contiguous (8 bytes each), slot i's memory address is computed in one arithmetic step: $\text{Address}(i) = \text{Base} + 8 \times i \implies \mathcal{O}(\mathbf{1})$.

```
# From src/data_structures/testaccess.py
starttime = time.time()
while count < 1000:
    x += lst[count]    # or lst[length//2 + count
     ]
    count += 1
print(length, time.time() - starttime)
```

Analysing Performance: $\mathcal{O}(1)$ Constant-Time Operations

Why Indexing and Length are $\mathcal{O}(1)$

- **Indexing (`L[i]`):** Because pointers are contiguous (8 bytes each), slot `i`'s memory address is computed in one arithmetic step: $\text{Address}(\texttt{i}) = \text{Base} + 8 \times \texttt{i} \implies \mathcal{O}(\mathbf{1})$.
- **Length (`len(L)`):** Reads `ob_size` directly from the header without counting elements $\implies \mathcal{O}(\mathbf{1})$.

```
1  # From src/data_structures/testaccess.py
2  starttime = time.time()
3  while count < 1000:
4      x += lst[count]    # or lst[length//2 + count
       ]
5      count += 1
6  print(length, time.time() - starttime)
```

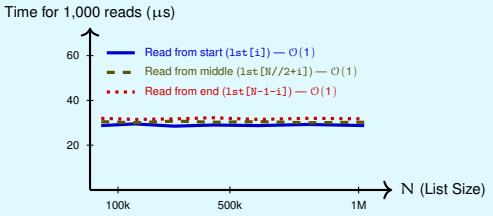
Analysing Performance: $\mathcal{O}(1)$ Constant-Time Operations

Why Indexing and Length are $\mathcal{O}(1)$

- **Indexing (`L[i]`):** Because pointers are contiguous (8 bytes each), slot `i`'s memory address is computed in one arithmetic step: $\text{Address}(i) = \text{Base} + 8 \times i \implies \mathcal{O}(\mathbf{1})$.
- **Length (`len(L)`):** Reads `ob_size` directly from the header without counting elements $\implies \mathcal{O}(\mathbf{1})$.

```
1  # From src/data_structures/testaccess.py
2  starttime = time.time()
3  while count < 1000:
4      x += lst[count]    # or lst[length//2 + count
       ]
5      count += 1
6  print(length, time.time() - starttime)
```

Empirical Benchmark (`testaccess.py`)

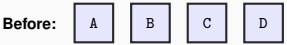


Access time ($\mu s = 10^{-6}$ s) is flat from 4,000 to 1,000,000 items!

The Cost of Insertion and Deletion: Maintaining Contiguity

The Trade-Off of Contiguous Memory

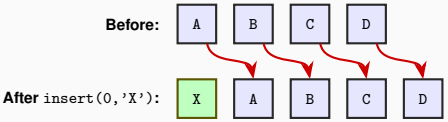
If we insert or delete an element in the beginning or middle of a dynamic array, **all subsequent pointers must be shifted** to keep the array contiguous and gap-free.



The Cost of Insertion and Deletion: Maintaining Contiguity

The Trade-Off of Contiguous Memory

If we insert or delete an element in the beginning or middle of a dynamic array, **all subsequent pointers must be shifted** to keep the array contiguous and gap-free.



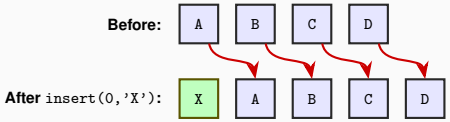
Computational Cost: $\mathcal{O}(N)$

Shifting N pointers requires N memory copy operations: **linear time** $\mathcal{O}(N)$.

The Cost of Insertion and Deletion: Maintaining Contiguity

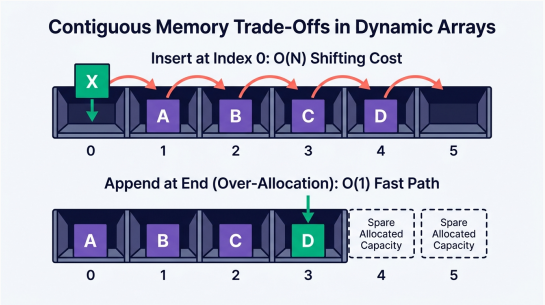
The Trade-Off of Contiguous Memory

If we insert or delete an element in the beginning or middle of a dynamic array, **all subsequent pointers must be shifted** to keep the array contiguous and gap-free.



Computational Cost: $\mathcal{O}(N)$

Shifting N pointers requires N memory copy operations: **linear time** $\mathcal{O}(N)$.



Complexity of Insertion and Deletion

Position Determines Complexity

- **Start (`insert(0,x)` / `pop(0)`):** Shifts all N items $\implies \mathcal{O}(\mathbf{N})$.

Complexity of Insertion and Deletion

Position Determines Complexity

- **Start (`insert(0,x)` / `pop(0)`):** Shifts all N items $\implies \mathcal{O}(\mathbf{N})$.
- **Middle (`insert(i,x)` / `pop(i)`):** Shifts $\mathrm{N} - \mathrm{i}$ items ($\approx \mathrm{N}/2$) $\implies \mathcal{O}(\mathbf{N})$.

Complexity of Insertion and Deletion

Position Determines Complexity

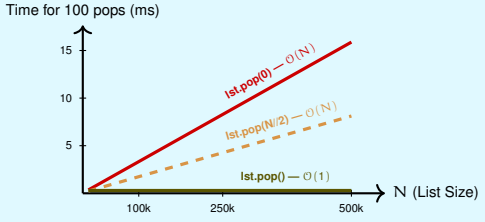
- **Start (`insert(0,x)` / `pop(0)`):** Shifts all N items $\implies \mathcal{O}(\mathbf{N})$.
- **Middle (`insert(i,x)` / `pop(i)`):** Shifts $\mathrm{N} - \mathrm{i}$ items ($\approx \mathrm{N}/2$) $\implies \mathcal{O}(\mathbf{N})$.
- **End (`pop()`):** Zero shifting $\implies \mathcal{O}(\mathbf{1})$.

Complexity of Insertion and Deletion

Position Determines Complexity

- **Start (`insert(0,x)` / `pop(0)`):** Shifts all N items $\implies \mathcal{O}(\mathbf{N})$.
- **Middle (`insert(i,x)` / `pop(i)`):** Shifts $\mathrm{N} - \mathrm{i}$ items ($\approx \mathrm{N}/2$) $\implies \mathcal{O}(\mathbf{N})$.
- **End (`pop()`):** Zero shifting $\implies \mathcal{O}(\mathbf{1})$.

Empirical Benchmark (`testdel_vs_size.py`)



Complexity of Insertion and Deletion

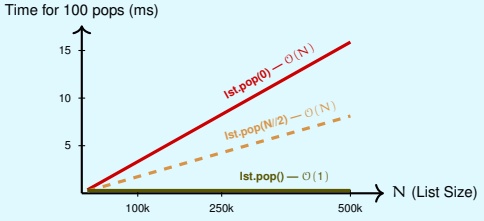
Position Determines Complexity

- **Start (`insert(0,x)` / `pop(0)`):** Shifts all N items $\implies \mathcal{O}(\mathbf{N})$.
- **Middle (`insert(i,x)` / `pop(i)`):** Shifts $N - i$ items ($\approx N/2$) $\implies \mathcal{O}(\mathbf{N})$.
- **End (`pop()`):** Zero shifting $\implies \mathcal{O}(\mathbf{1})$.

The $\mathcal{O}(N^2)$ Trap (`testdel.py`)

Removing all N items from the front via `lst.pop(0)` shifts $(N-1) + \cdots + 1 = \frac{N(N-1)}{2} = \mathcal{O}(\mathbf{N^2})$ pointers!
(Week 3 introduces Queues with $\mathcal{O}(1)$ front removal.)

Empirical Benchmark (`testdel_vs_size.py`)



The Complexity of `.append()` and Growth Strategies

Fast Path vs. Slow Path

- **Fast Path (`ob_size < allocated`):** Write pointer into next spare slot and increment `ob_size`. Takes $\mathcal{O}(\mathbf{1})$ time.

The Complexity of `.append()` and Growth Strategies

Fast Path vs. Slow Path

- **Fast Path (`ob_size < allocated`):** Write pointer into next spare slot and increment `ob_size`. Takes $\mathcal{O}(\mathbf{1})$ time.
- **Slow Path (`ob_size == allocated`):** Array is full! Allocate a larger array, copy all N pointers, and free the old array. Takes $\mathcal{O}(\mathbf{N})$ time.

The Complexity of `.append()` and Growth Strategies

Fast Path vs. Slow Path

- **Fast Path (`ob_size < allocated`):** Write pointer into next spare slot and increment `ob_size`. Takes $\mathcal{O}(\mathbf{1})$ time.
- **Slow Path (`ob_size == allocated`):** Array is full! Allocate a larger array, copy all N pointers, and free the old array. Takes $\mathcal{O}(\mathbf{N})$ time.

Naive Strategy: Additive Growth ($+C$)

If we only add a fixed constant (e.g. $+1$ or $+10$ slots) on each resize, resizes happen every C steps, costing $C + 2C + 3C + \cdots + N = \mathcal{O}(\mathbf{N^2})$ total time for N appends!

The Complexity of `.append()` and Growth Strategies

Fast Path vs. Slow Path

- **Fast Path (`ob_size < allocated`):** Write pointer into next spare slot and increment `ob_size`. Takes $\mathcal{O}(\mathbf{1})$ time.
- **Slow Path (`ob_size == allocated`):** Array is full! Allocate a larger array, copy all N pointers, and free the old array. Takes $\mathcal{O}(\mathbf{N})$ time.

Naive Strategy: Additive Growth ($+C$)

If we only add a fixed constant (e.g. $+1$ or $+10$ slots) on each resize, resizes happen every C steps, costing $C + 2C + 3C + \cdots + N = \mathcal{O}(\mathbf{N^2})$ total time for N appends!

Geometric Growth ($\times g$)

Multiply capacity by a constant growth factor $g > 1$ whenever full. Resizes occur **exponentially less often** as N grows!

The Complexity of `.append()` and Growth Strategies

Fast Path vs. Slow Path

- **Fast Path (`ob_size < allocated`):** Write pointer into next spare slot and increment `ob_size`. Takes $\mathcal{O}(\mathbf{1})$ time.
- **Slow Path (`ob_size == allocated`):** Array is full! Allocate a larger array, copy all N pointers, and free the old array. Takes $\mathcal{O}(\mathbf{N})$ time.

Naive Strategy: Additive Growth ($+\mathrm{C}$)

If we only add a fixed constant (e.g. $+1$ or $+10$ slots) on each resize, resizes happen every C steps, costing $\mathrm{C} + 2\mathrm{C} + 3\mathrm{C} + \cdots + \mathrm{N} = \mathcal{O}(\mathbf{N^2})$ total time for N appends!

Geometric Growth ($\times \mathrm{g}$)

Multiply capacity by a constant growth factor $\mathrm{g} > 1$ whenever full. Resizes occur **exponentially less often** as N grows!

CPython's Growth Factor: $\mathrm{g} \approx \frac{9}{8} = 1.125$

Why does CPython grow by $\approx 12.5\%$ instead of doubling ($\mathrm{g} = 2$)?

- **Doubling ($\mathrm{g} = 2$):** Wastes up to 50% of allocated memory right after a resize.
- **CPython ($\mathrm{g} = 1.125$):** Wastes at most $\mathbf{12.5}\%$ spare memory while still guaranteeing $\mathcal{O}(1)$ amortised append time!

Amortised Analysis of Geometric Growth

Aggregate Analysis Over N Appends (`testappend.py`)

Amortised analysis averages the occasional $\mathcal{O}(N)$ resize over the many $\mathcal{O}(1)$ appends:

- With growth factor $g > 1$, the k resizes up to size N ($g^k \leqslant N$) copy $1, g, g^2, \ldots, g^k$ pointers.
- Their sum ($\sum$) is a **geometric series**:

$$C(N) = \sum_{i=0}^{k} g^i = \frac{g^{k+1} - 1}{g - 1} < \frac{g}{g - 1} N$$

Amortised Analysis of Geometric Growth

Aggregate Analysis Over N Appends (`testappend.py`)

Amortised analysis averages the occasional $\mathcal{O}(N)$ resize over the many $\mathcal{O}(1)$ appends:

- With growth factor $g > 1$, the k resizes up to size N ($g^k \leqslant N$) copy $1, g, g^2, \ldots, g^k$ pointers.
- Their sum ($\sum$) is a **geometric series**:

$$C(N) = \sum_{i=0}^{k} g^i = \frac{g^{k+1} - 1}{g - 1} < \frac{g}{g - 1} N$$

- **Doubling ($g = 2$):** $C(N) < \frac{2}{2-1} N = 2N$ copies ($+N$ writes $< 3N$).
- **CPython ($g = \frac{9}{8}$):** $C(N) < \frac{9/8}{1/8} N = 9N$ copies ($+N$ writes $< 10N$).

Amortised Analysis of Geometric Growth

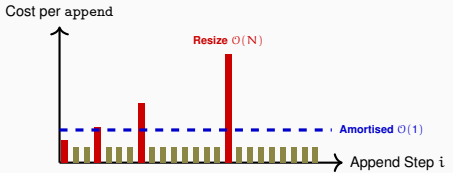
Aggregate Analysis Over N Appends (`testappend.py`)

Amortised analysis averages the occasional $\mathcal{O}(N)$ resize over the many $\mathcal{O}(1)$ appends:

- With growth factor $g > 1$, the k resizes up to size N ($g^k \leqslant N$) copy $1, g, g^2, \ldots, g^k$ pointers.
- Their sum ($\sum$) is a **geometric series**:

$$C(N) = \sum_{i=0}^{k} g^i = \frac{g^{k+1} - 1}{g - 1} < \frac{g}{g - 1} N$$

- **Doubling ($g = 2$):** $C(N) < \frac{2}{2-1} N = 2N$ copies ($+N$ writes $< 3N$).
- **CPython ($g = \frac{9}{8}$):** $C(N) < \frac{9/8}{1/8} N = 9N$ copies ($+N$ writes $< 10N$).



Amortised Complexity

Total work for N appends is $\mathcal{O}(N)$, so the average cost per append is $\frac{\mathcal{O}(N)}{N} = \mathcal{O}(\mathbf{1})$ (Amortised).

Searching Lists: Linear vs. Binary Search

List Performance Summary and the Search Problem

List Operations Cheat Sheet

Operation	Code	Average
Indexing	`L[i]`	$\mathcal{O}(1)$
Length	`len(L)`	$\mathcal{O}(1)$
Append (End)	`L.append(x)`	$\mathcal{O}(1)$ (Amort.)
Pop (End)	`L.pop()`	$\mathcal{O}(1)$
Insert/Pop (Start)	`L.pop(0)`	$\mathcal{O}(\mathrm{N})$
Linear Search	`x in L`	$\mathcal{O}(\mathbf{N})$

List Performance Summary and the Search Problem

List Operations Cheat Sheet

Operation	Code	Average
Indexing	`L[i]`	$\mathcal{O}(1)$
Length	`len(L)`	$\mathcal{O}(1)$
Append (End)	`L.append(x)`	$\mathcal{O}(1)$ (Amort.)
Pop (End)	`L.pop()`	$\mathcal{O}(1)$
Insert/Pop (Start)	`L.pop(0)`	$\mathcal{O}(N)$
Linear Search	`x in L`	$\mathcal{O}(\mathbf{N})$

Why Unsorted Search is $\mathcal{O}(N)$

Without knowing where values live, checking `target in lst` must scan elements sequentially from index `0` to `N-1`.

List Performance Summary and the Search Problem

List Operations Cheat Sheet

Operation	Code	Average
Indexing	`L[i]`	$\mathcal{O}(1)$
Length	`len(L)`	$\mathcal{O}(1)$
Append (End)	`L.append(x)`	$\mathcal{O}(1)$ (Amort.)
Pop (End)	`L.pop()`	$\mathcal{O}(1)$
Insert/Pop (Start)	`L.pop(0)`	$\mathcal{O}(\mathrm{N})$
Linear Search	`x in L`	$\mathcal{O}(\mathbf{N})$

Why Unsorted Search is $\mathcal{O}(\mathrm{N})$

Without knowing where values live, checking `target in lst` must scan elements sequentially from index `0` to `N-1`.

Linear Search (`isin.py`)

```
1  def test_is_in():
2      assert is_in([1, 2, 3], 3)
3      assert not is_in([1, 2, 3], 4)
4
5  def is_in(lst, target):
6      for value in lst:
7          if value == target:
8              return True
9      return False
```

- **Best Case:** $\mathcal{O}(1)$ (at index `0`).
- **Worst Case:** N comparisons ($\mathcal{O}(\mathrm{N})$).
- **Average Case:** $\mathrm{N}/2$ comparisons ($\mathcal{O}(\mathrm{N})$).

Exploiting Order: Binary Search on a Sorted List

The Bisection Idea (`binarysearch.py`)

If the list is **sorted** ascending, halve the search space:

1. Maintain half-open interval `[start, end)` (`start` included, `end` excluded).
2. Midpoint via **floor division** (`//` rounds down): `middle = (start + end) // 2`.
3. If `val >= lst[middle]`, set `start = middle`; else `end = middle`.
4. Stop at `end - start == 1`; check `lst[start] == val`.

Exploiting Order: Binary Search on a Sorted List

The Bisection Idea (`binarysearch.py`)

If the list is **sorted** ascending, halve the search space:

1. Maintain half-open interval `[start, end)` (`start` included, `end` excluded).
2. Midpoint via **floor division** (`//` rounds down): `middle = (start + end) // 2`.
3. If `val >= lst[middle]`, set `start = middle`; else `end = middle`.
4. Stop at `end - start == 1`; check `lst[start] == val`.

Implementation (`binarysearch.py`)

```
1  def is_in_bisect(lst, val):
2      if len(lst) == 0:
3          return False
4      start, end = 0, len(lst)  # [start, end)
5      while end - start > 1:
6          middle = (start + end) // 2
7          if val >= lst[middle]:
8              start = middle
9          else:
10             end = middle
11     return lst[start] == val
```

Exploiting Order: Binary Search on a Sorted List

The Bisection Idea (`binarysearch.py`)

If the list is **sorted** ascending, halve the search space:

1. Maintain half-open interval `[start, end)` (`start` included, `end` excluded).
2. Midpoint via **floor division** (`//` rounds down): `middle = (start + end) // 2`.
3. If `val >= lst[middle]`, set `start = middle`; else `end = middle`.
4. Stop at `end - start == 1`; check `lst[start] == val`.

Boundary Unit Tests

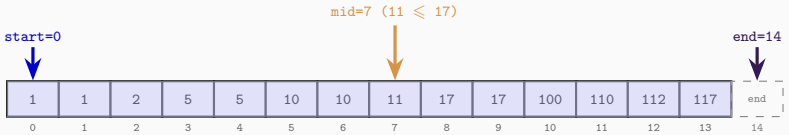
```
1  assert not is_in_bisect([], 4)
2  assert is_in_bisect([1], 1)
3  assert is_in_bisect([1, 2, 2, 2, 3], 2)
```

Implementation (`binarysearch.py`)

```
1  def is_in_bisect(lst, val):
2      if len(lst) == 0:
3          return False
4      start, end = 0, len(lst)  # [start, end)
5      while end - start > 1:
6          middle = (start + end) // 2
7          if val >= lst[middle]:
8              start = middle
9          else:
10             end = middle
11     return lst[start] == val
```

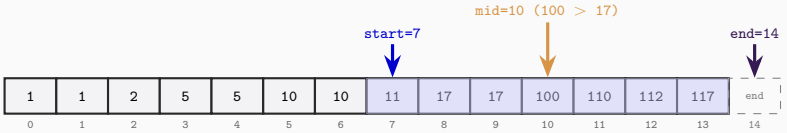
Visualising Binary Search ($\mathcal{O}(\log N)$)

Tracing `is_in_bisect(seq, 17)` from `src/data_structures/binarysearch.py`:



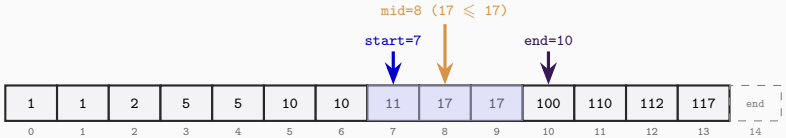
Visualising Binary Search ($\mathcal{O}(\log N)$)

Tracing `is_in_bisect(seq, 17)` from `src/data_structures/binarysearch.py`:



Visualising Binary Search ($\mathcal{O}(\log N)$)

Tracing `is_in_bisect(seq, 17)` from `src/data_structures/binarysearch.py`:



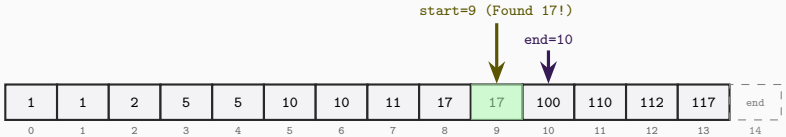
Visualising Binary Search ($\mathcal{O}(\log N)$)

Tracing `is_in_bisect(seq, 17)` from `src/data_structures/binarysearch.py`:



Visualising Binary Search ($\mathcal{O}(\log \mathrm{N})$)

Tracing `is_in_bisect(seq, 17)` from `src/data_structures/binarysearch.py`:



Logarithmic Time: $\mathcal{O}(\log \mathrm{N})$

Halving N repeatedly reaches size 1 in $\approx \log_2 \mathrm{N}$ steps: for $\mathrm{N} = 1{,}000{,}000$, Binary Search needs at most $\mathbf{20}$ comparisons instead of 1,000,000!

The Sorted List Trade-Off

Binary search requires the list to be **sorted**. Inserting a new item into a sorted list still requires shifting elements ($\mathcal{O}(\mathbf{N})$).

Hash Mappings: Dictionaries and Sets

Beyond $\mathcal{O}(\log N)$: The Quest for $\mathcal{O}(1)$ Lookup

The Engineering Challenge

What if our dataset is both **huge** and **constantly changing** (e.g. validating 1,000,000 transit cards or active usernames)?

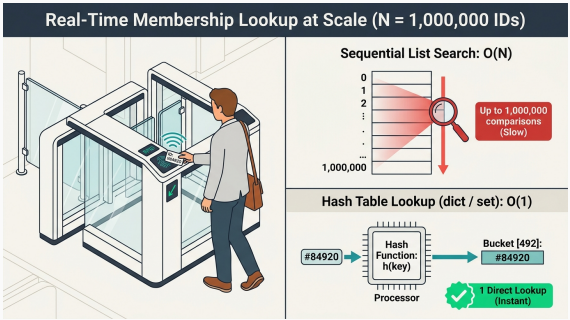
- **Unsorted `list`:** Fast $\mathcal{O}(1)$ append, slow $\mathcal{O}(N)$ search.

Beyond $\mathcal{O}(\log N)$: The Quest for $\mathcal{O}(1)$ Lookup

The Engineering Challenge

What if our dataset is both **huge** and **constantly changing** (e.g. validating 1,000,000 transit cards or active usernames)?

- **Unsorted** `list`**:** Fast $\mathcal{O}(1)$ append, slow $\mathcal{O}(N)$ search.
- **Sorted** `list`**:** Fast $\mathcal{O}(\log N)$ binary search, slow $\mathcal{O}(N)$ insert shift.



Beyond $\mathcal{O}(\log N)$: The Quest for $\mathcal{O}(1)$ Lookup

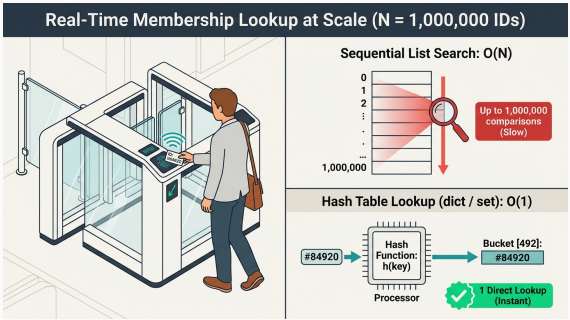
The Engineering Challenge

What if our dataset is both **huge** and **constantly changing** (e.g. validating 1,000,000 transit cards or active usernames)?

- **Unsorted `list`:** Fast $\mathcal{O}(1)$ append, slow $\mathcal{O}(N)$ search.
- **Sorted `list`:** Fast $\mathcal{O}(\log N)$ binary search, slow $\mathcal{O}(N)$ insert shift.

The Solution: Hash Tables

Hash Tables achieve $\mathcal{O}(\mathbf{1})$ average time for *both* lookup and insertion, forming the engine behind Python's `dict` and `set`.

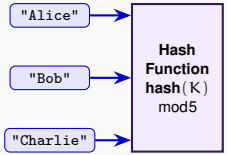


How Hashing Works Conceptually

1. The Hash Function (`hash(K)`)

Maps an arbitrary key K (e.g. `"Alice"`) to a fixed-size integer in $\mathcal{O}(1)$ time:

- **Deterministic:** Identical keys produce the exact same hash within a program run.
- **Uniform:** Spreads keys evenly across integers.



How Hashing Works Conceptually

1. The Hash Function (`hash(K)`)

Maps an arbitrary key K (e.g. `"Alice"`) to a fixed-size integer in $\mathcal{O}(1)$ time:

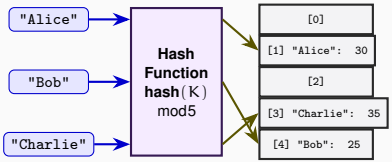
- **Deterministic:** Identical keys produce the exact same hash within a program run.
- **Uniform:** Spreads keys evenly across integers.

2. Modulo Bucket Mapping (mod)

Given M buckets, we use the **modulo operator** (mod, or **%** in Python, which returns the remainder after division by M):

$$\text{Index} = \text{hash}(K) \bmod M \in \{0, \ldots, M-1\}$$

We jump directly to `buckets[Index]` in $\mathcal{O}(1)$ time!



How Hashing Works Conceptually

1. The Hash Function (`hash(K)`)

Maps an arbitrary key K (e.g. `"Alice"`) to a fixed-size integer in $\mathcal{O}(1)$ time:

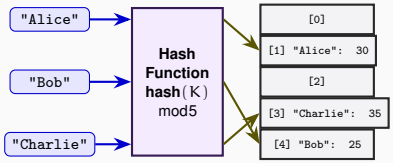
- **Deterministic:** Identical keys produce the exact same hash within a program run.
- **Uniform:** Spreads keys evenly across integers.

2. Modulo Bucket Mapping (mod)

Given M buckets, we use the **modulo operator** (mod, or `%` in Python, which returns the remainder after division by M):

$$\text{Index} = \text{hash}(K) \bmod M \in \{0, \ldots, M-1\}$$

We jump directly to `buckets[Index]` in $\mathcal{O}(1)$ time!



Collisions & Resizing (Preview of Week 4)

When two keys land on the same index (a **collision**), the table resolves it and resizes when full (amortised $\mathcal{O}(1)$). We will build a `HashTable` from scratch in Week 4!

Dictionaries (`dict`): Key-Value Mappings

Interface and $\mathcal{O}(1)$ Operations

A `dict` is a mutable mapping from unique **keys** to **values**:

```
# Mapping names (keys) to ages (values)
ages = {"Alice": 30, "Bob": 25, "Charlie": 35}

# Lookup by key: O(1) average
bobs_age = ages["Bob"]        # 25

# Insert or update: O(1) amortised
ages["David"] = 40

# Key membership check: O(1) average
is_present = "Alice" in ages  # True
```

Dictionaries (`dict`): Key-Value Mappings

Interface and $\mathcal{O}(1)$ Operations

A `dict` is a mutable mapping from unique **keys** to **values**:

```python
# Mapping names (keys) to ages (values)
ages = {"Alice": 30, "Bob": 25, "Charlie": 35}

# Lookup by key: O(1) average
bobs_age = ages["Bob"]          # 25

# Insert or update: O(1) amortised
ages["David"] = 40

# Key membership check: O(1) average
is_present = "Alice" in ages  # True
```

The Hashability Requirement

Keys must be **hashable** (immutable value):

- **Hashable:** `int`, `float`, `str`, and `tuple` *(if all its elements are hashable!)*.
- **Unhashable:** `list`, `dict`, `set`.

```python
grid = {(0, 1): "wall"} # Valid tuple key
bad  = {[0, 1]: "wall"} # TypeError:
     unhashable 'list'
```

Sets (`set`): Unique Elements and Fast Membership

Definition and Usage

A `set` is an **unordered**, mutable collection of **unique**, hashable elements:

```python
# Duplicates are removed automatically
colours = {"red", "green", "blue", "red"}
# colours == {"red", "green", "blue"}

# Fast O(1) average membership testing
has_red = "red" in colours    # True

# Adding and removing elements: O(1) average
colours.add("yellow")
colours.remove("green")
```

Note: Use `set()` for an empty set ({} is an empty `dict`!).

Sets (`set`): Unique Elements and Fast Membership

Definition and Usage

A `set` is an **unordered**, mutable collection of **unique**, hashable elements:

```python
# Duplicates are removed automatically
colours = {"red", "green", "blue", "red"}
# colours == {"red", "green", "blue"}

# Fast O(1) average membership testing
has_red = "red" in colours    # True

# Adding and removing elements: O(1) average
colours.add("yellow")
colours.remove("green")
```

Note: Use `set()` for an empty set ({} is an empty `dict`!).

Under the Hood

A `set` is a hash table storing **keys only** (no value pointers), giving $\mathcal{O}(1)$ average lookup and insertion.

Sets (`set`): Unique Elements and Fast Membership

Definition and Usage

A `set` is an **unordered**, mutable collection of **unique**, hashable elements:

```python
# Duplicates are removed automatically
colours = {"red", "green", "blue", "red"}
# colours == {"red", "green", "blue"}

# Fast O(1) average membership testing
has_red = "red" in colours    # True

# Adding and removing elements: O(1) average
colours.add("yellow")
colours.remove("green")
```

Note: Use `set()` for an empty set ({} is an empty `dict`!).

Under the Hood

A `set` is a hash table storing **keys only** (no value pointers), giving $\mathcal{O}(1)$ average lookup and insertion.

Engineering Use Cases

1. **Fast Membership:** Converting 10^6 IDs to a `set` turns $\mathcal{O}(N)$ scans into $\mathcal{O}(1)$ lookups.
2. **Deduplicating a List in $\mathcal{O}(N)$:**
   - `list(set(L))`: Fast, **loses order**.
   - `list(dict.fromkeys(L))`: Fast, **preserves order**.

Conclusion

Summary of Core Data Structures and Key Takeaways

Type	Underlying Implementation	Ordered?	Mutable?	Key Performance Characteristics
`list` (unsorted)	Dynamic Array of Pointers	Yes	Yes	$\mathcal{O}(1)$ index/append; $\mathcal{O}(N)$ insert(0)/pop(0)/search
`list` (sorted)	Dynamic Array of Pointers	Yes	Yes	$\mathcal{O}(\log N)$ binary search (`binarysearch.py`); $\mathcal{O}(N)$ insert
`dict`	Hash Table (Key $\rightarrow$ Value)	Yes (3.7+)	Yes (Keys immut.)	$\mathcal{O}(1)$ average lookup, insert, delete
`set`	Hash Table (Keys only)	No	Yes (Elems immut.)	$\mathcal{O}(1)$ average membership (`x in S`), add, remove
`tuple`	Fixed-Size Pointer Array	Yes	No	$\mathcal{O}(1)$ index; hashable if all elements are hashable

Summary of Core Data Structures and Key Takeaways

Type	Underlying Implementation	Ordered?	Mutable?	Key Performance Characteristics
`list` (unsorted)	Dynamic Array of Pointers	Yes	Yes	$\mathcal{O}(1)$ index/append; $\mathcal{O}(N)$ insert(0)/pop(0)/search
`list` (sorted)	Dynamic Array of Pointers	Yes	Yes	$\mathcal{O}(\log N)$ binary search (`binarysearch.py`); $\mathcal{O}(N)$ insert
`dict`	Hash Table (Key $\rightarrow$ Value)	Yes (3.7+)	Yes (Keys immut.)	$\mathcal{O}(1)$ average lookup, insert, delete
`set`	Hash Table (Keys only)	No	Yes (Elems immut.)	$\mathcal{O}(1)$ average membership (`x in S`), add, remove
`tuple`	Fixed-Size Pointer Array	Yes	No	$\mathcal{O}(1)$ index; hashable if all elements are hashable

Core Principles

- **Memory Model:** Variables are references; watch out for aliasing and shallow copies of nested lists.
- **Amortisation:** Geometric over-allocation ($g \approx 1.125$) makes `list.append()` $\mathcal{O}(1)$ on average.

Summary of Core Data Structures and Key Takeaways

Type	Underlying Implementation	Ordered?	Mutable?	Key Performance Characteristics
`list` (unsorted)	Dynamic Array of Pointers	Yes	Yes	$\mathcal{O}(1)$ index/append; $\mathcal{O}(N)$ insert(0)/pop(0)/search
`list` (sorted)	Dynamic Array of Pointers	Yes	Yes	$\mathcal{O}(\log N)$ binary search (`binarysearch.py`); $\mathcal{O}(N)$ insert
`dict`	Hash Table (Key $\rightarrow$ Value)	Yes (3.7+)	Yes (Keys immut.)	$\mathcal{O}(1)$ average lookup, insert, delete
`set`	Hash Table (Keys only)	No	Yes (Elems immut.)	$\mathcal{O}(1)$ average membership (`x in S`), add, remove
`tuple`	Fixed-Size Pointer Array	Yes	No	$\mathcal{O}(1)$ index; hashable if all elements are hashable

Core Principles

- **Memory Model:** Variables are references; watch out for aliasing and shallow copies of nested lists.
- **Amortisation:** Geometric over-allocation ($g \approx 1.125$) makes `list.append()` $\mathcal{O}(1)$ on average.

The Engineering Trade-Off

No single data structure is best for every task. Choose based on your workload's dominant operations (ordered indexing vs. frequent membership lookups).